

Lecture 03: Value vs. Reference and JavaScript Operators

Primitive vs. Non-Primitive Data Types:

1. Copy by Value vs. Copy by Reference

Primitive Data Types (Copy by Value): When a primitive variable is assigned to another, a completely new copy of the value is created in memory.

```
let firstNumber = 10;
let secondNumber = firstNumber;
firstNumber = 20;
```



secondNumber receives an independent copy of 10. Changing firstNumber later does not affect secondNumber.

Non-Primitive Types (Copy by Reference): Non-primitive variables (like Objects and Arrays) just store a memory address (reference).

```
let obj1 = {
  name: "Rohit",
  age: 20
};
let obj2 = obj1;
```

Both obj1 and obj2 now point to the exact same object in memory. Modifying data through obj2 will reflect in obj1.

Why this design?

It is an optimization rule created by **ECMAScript** (the governing body that designs JavaScript). Primitives are small and fixed in size, so copying them is cheap. Non-primitives (like large arrays or objects) take up a lot of space, so copying the whole object every time would crash the browser. Instead, JS just passes around the lightweight memory address.

2. Comparison: Value vs. Reference

Primitive Comparison: JavaScript compares the actual data value.

```
let a = 10;
let b = 10;
console.log(a == b);
```

Non-Primitive Comparison: JavaScript compares the memory addresses, not the internal data.

```
let obj1 = {
  name: "Rohit",
  age: 20
};
let obj2 = {
  name: "Rohit",
  age: 20
};
console.log(obj1 == obj2);
```

3. Mutability

Primitives are Immutable: The actual value in memory cannot be altered. Variables can be reassigned to completely new values, but the original value itself remains unchanged.

```
let a = 10;  
a = 20;
```



Non-Primitives are Mutable: The internal contents (properties or array elements) of an object can be modified directly.

```
let obj1 = {  
  name: "Rohit",  
  age: 20  
};  
let obj2 = obj1;  
obj2.name = "Aditya";
```

The *const* Behavior:

- **With Primitives:** The value is locked.
- **With Non-Primitives:** The memory address is locked, but the internal contents remain entirely mutable.

The *var* Keyword (Legacy Method):

Before modern JavaScript (let and const), variables were declared using var. It has two unique behaviors that can lead to bugs, which is why its use is currently discouraged.

1. Hoisting

Variables declared with var are "hoisted" to the top of their scope during the compilation phase. They are initialized with a default value of undefined.

```
console.log(a);  
var a = 10;
```

2. Lack of Block Scope

Unlike let and const, var does not respect block scope (e.g., variables created inside if statements or for loops). It is either scoped to the nearest enclosing function or globally scoped.

```
if(true) {  
  var a = 10;  
}  
console.log(a);
```

Operators:

Operators are symbols or keywords used to perform operations on values (operands).

1. Arithmetic Operators

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus
**	Exponentiation

2. Assignment Operators

Operator	Meaning
=	Assign value
+=	Add and assign
-=	Subtract and assign
*=	Multiply and assign
/=	Divide and assign
%=	Modulus and assign
**=	Exponent and assign

3. Increment and Decrement Operators

- They are **Unary Operators** because they operate on one variable only.
- They are used to increase or decrease a value by 1.

Operator	Meaning
++	Increment
--	Decrement

Pre-Increment / Pre-Decrement

Value is changed first, then used in the expression.

Post-Increment / Post-Decrement

The current value is used in the expression first, then changed.

4. Comparison Operators

Operator	Meaning
==	Equal (value only)
===	Strict equal (value + type)
!=	Not equal
!==	Strict not equal
>	Greater Than
<	Less Than
>=	Greater or equal
<=	Less or equal

5. Logical Operators

Operator	Meaning
&&	Logical AND
	Logical OR
!	Logical Not

Short-Circuiting:

- **&& (Logical AND):** Evaluates left to right. Returns the first falsy value it encounters. If everything is truthy, it returns the last value.
- **|| (Logical OR):** Evaluates left to right. Returns the first truthy value it encounters. If everything is falsy, it returns the last value.

The Weird Rules of *null*:

- *null* is loosely equal to *undefined* only.
- For relational comparison operators (>, <, >=, <=), JavaScript converts *null* to 0.

Best Practice: Don't memorize these weird JS coercion rules. Avoid writing code that relies on tricky comparisons between different data types!

Note: Data retrieved from an HTML form always comes back as a String.

What is Type Conversion?

Type conversion is the process of converting a value from one data type to another (e.g., converting a String to a Number). In JavaScript, this occurs in two distinct ways:

- **Explicit Conversion (Type Casting):** The developer manually converts the data type using built-in JavaScript functions.
- **Implicit Conversion (Type Coercion):** The JavaScript engine automatically converts the data type behind the scenes during operations.

Explicit Conversion (Manual Casting):

1. Converting to a Number

- `Number(value)`: Converts a string or boolean into a strict number. If the string contains non-numeric characters, it returns NaN (Not a Number).
- `parseInt(value)`: Extracts the integer portion of a string.
- `parseFloat(value)`: Extracts the floating-point (decimal) portion of a string.
- **Unary Plus (+) Operator:** A shorthand trick to explicitly cast a string to a number.

2. Converting to a String

- `String(value)`: Converts any value into a string.
- `.toString()`: A method available on numbers and booleans. It cannot be used on null or undefined.

3. Converting to a Boolean

The `Boolean()` function converts values to either true or false. This relies on the concept of "Truthy" and "Falsy" values.

Implicit Conversion (Automatic Coercion):

1. String Coercion (The + Operator)

When the + operator is used with a string, the other operand is automatically converted to a string, and concatenation occurs.

2. Numeric Coercion (The -, *, /, % Operators)

When mathematical operators (other than +) are used, strings are automatically converted into numbers.